

Intuitive Explanations · Real-World Examples · Rich Visualizations

Session 3 — Scaler Academy

Train-Test Split

Standardization

One-Hot Encoding

Outliers

Multicollinearity

VIF

5 Assumptions

Polynomial Regression

Bias-Variance

Regularization

✂️ CONCEPT 1 — Train-Test Split

When building a model, you **never test it on the same data you trained it with**. That would be like giving a student the exact exam paper they studied — of course they'd score 100%! The real question is: how does the model perform on *data it has never seen before*?

■ The Problem: If we train and test on the SAME data, R^2 will always look great — but the model might fail on real-world data!

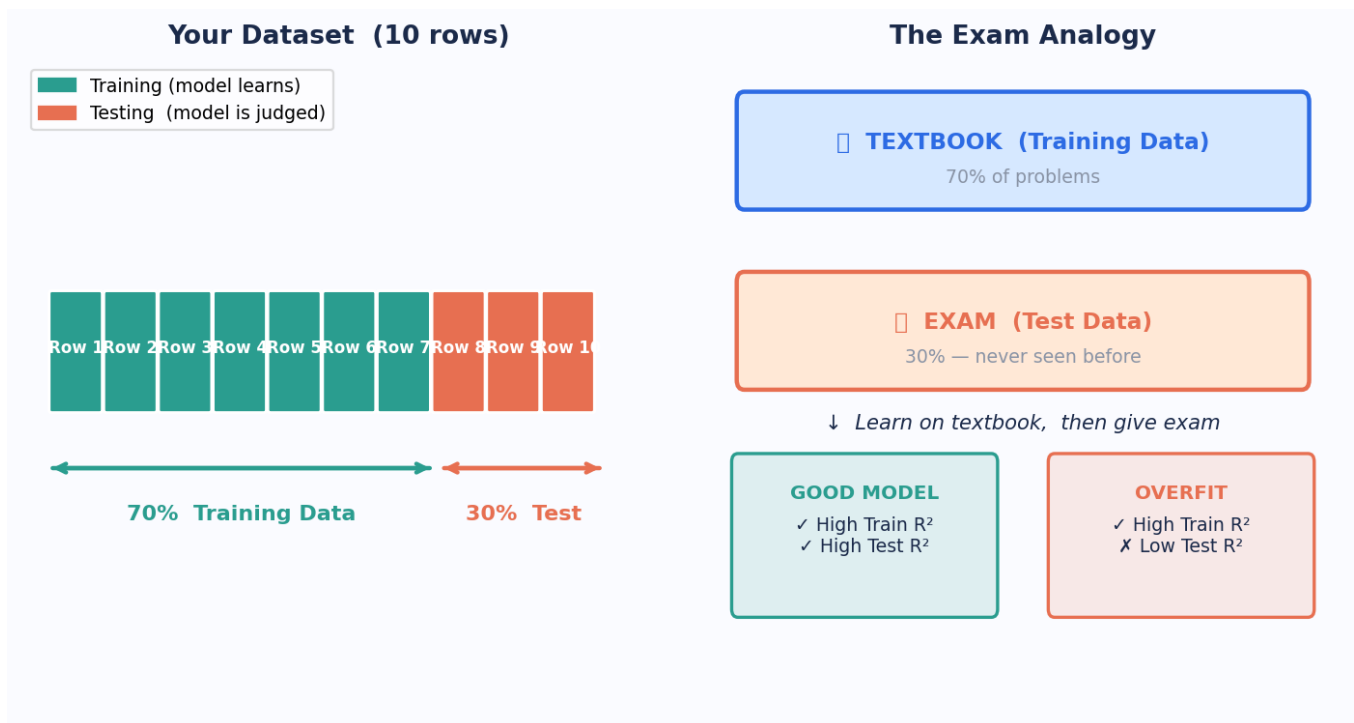


Figure 1: Train-Test Split — the Exam Analogy

Split Ratio	When to Use	Why
80 – 20	Small datasets (<5k rows)	Keep max data for training
70 – 30	Medium/large datasets	More reliable test evaluation
60 – 20 – 20	Large + need validation set	Tune hyperparameters separately

■ Always use `random_state` in `train_test_split()` to make experiments reproducible. Same seed = same split every time.

■■ CONCEPT 2 — Why Standardization / Scaling?

Imagine you are predicting **car selling price** using two features: **km_driven** (ranges 10,000 – 200,000) and **mileage** (ranges 10 – 30 km/l). The first feature has values 10,000× larger than the second. The model's math will naturally assign a tiny weight to km_driven just to "cancel" its large numbers — leading to biased, misleading coefficients.

■ Problem: km_driven = 150,000 vs mileage = 18. The model sees km_driven as 8,000× more "important" just because of its unit — NOT because it actually matters more!

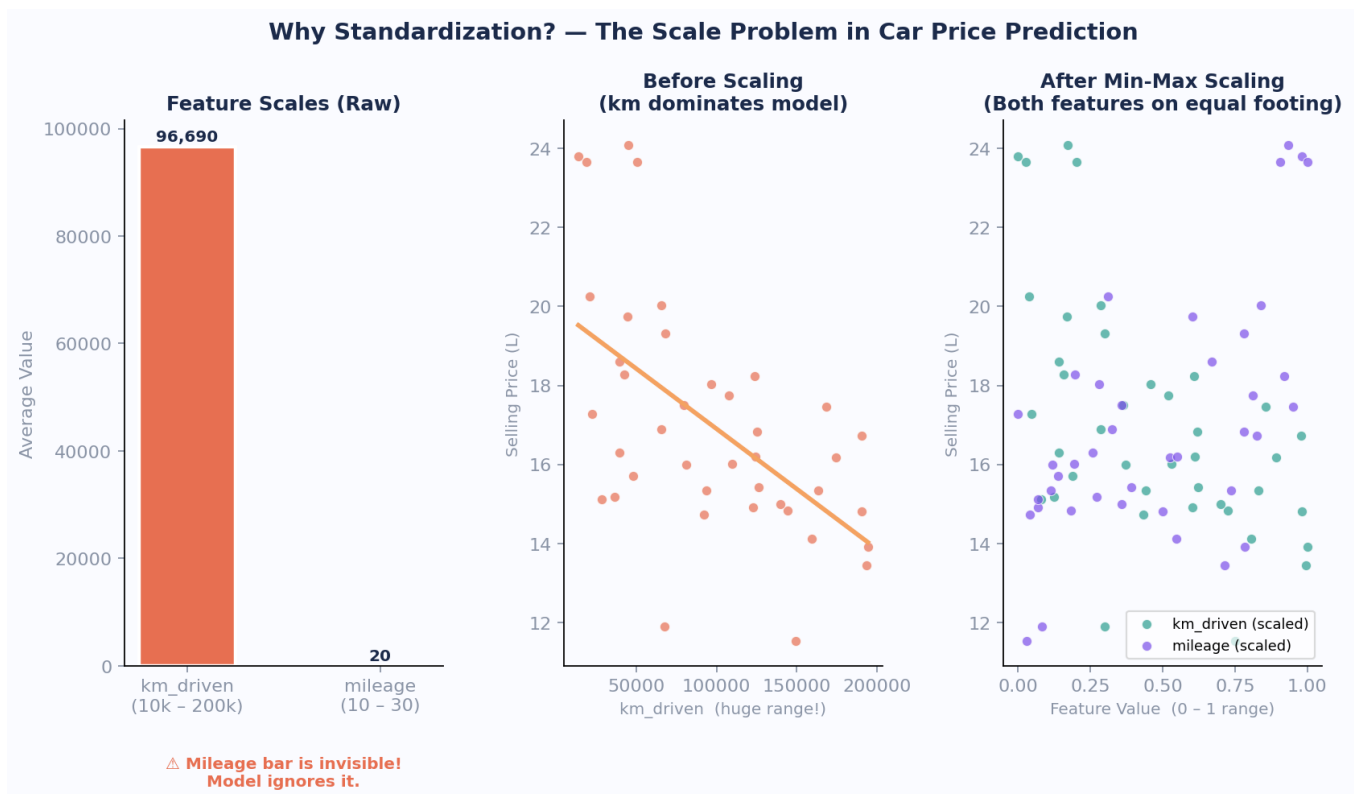


Figure 2: Feature scale mismatch and the fix — Min-Max Scaling

Min-Max Scaler — Bringing Everything to [0, 1]

The formula is elegantly simple: subtract the minimum and divide by the range. Every feature now lives between 0 and 1, regardless of its original unit.

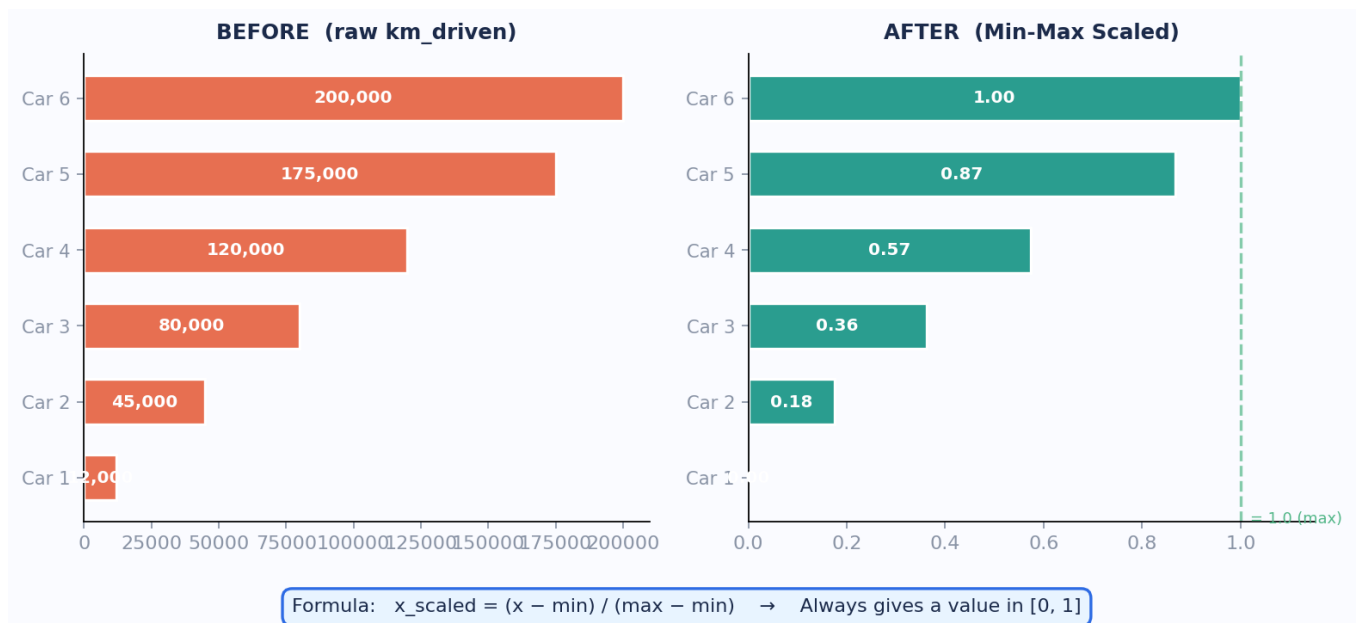


Figure 3: Before vs After Min-Max Scaling on km_driven

Method	Formula	Best For	Watch Out For
Min-Max Scaler	$x_s = (x - \min) / (\max - \min)$	Neural nets, KNN, SVM	Sensitive to outliers!
Z-Score (Standard)	$x_s = (x - \text{mean}) / \text{std}$	Linear / Logistic Reg.	Assumes near-normal dist.

■ Fit the scaler on TRAINING data only. Then transform both train and test. Never fit on test — that would be data leakage!

■ CONCEPT 3 — One-Hot Encoding — Teaching Math to Read Categories

A linear regression equation works with **numbers**. But features like *origin* (American / European / Asian) are **text categories**. The naive fix — assigning 1, 2, 3 — accidentally tells the model that "Asian (3) is twice as much as European (2)" which is complete nonsense!

■ The Ordering Trap: If America=1, Europe=2, Asia=3, the model wrongly assumes Europe is the "average" of America and Asia!

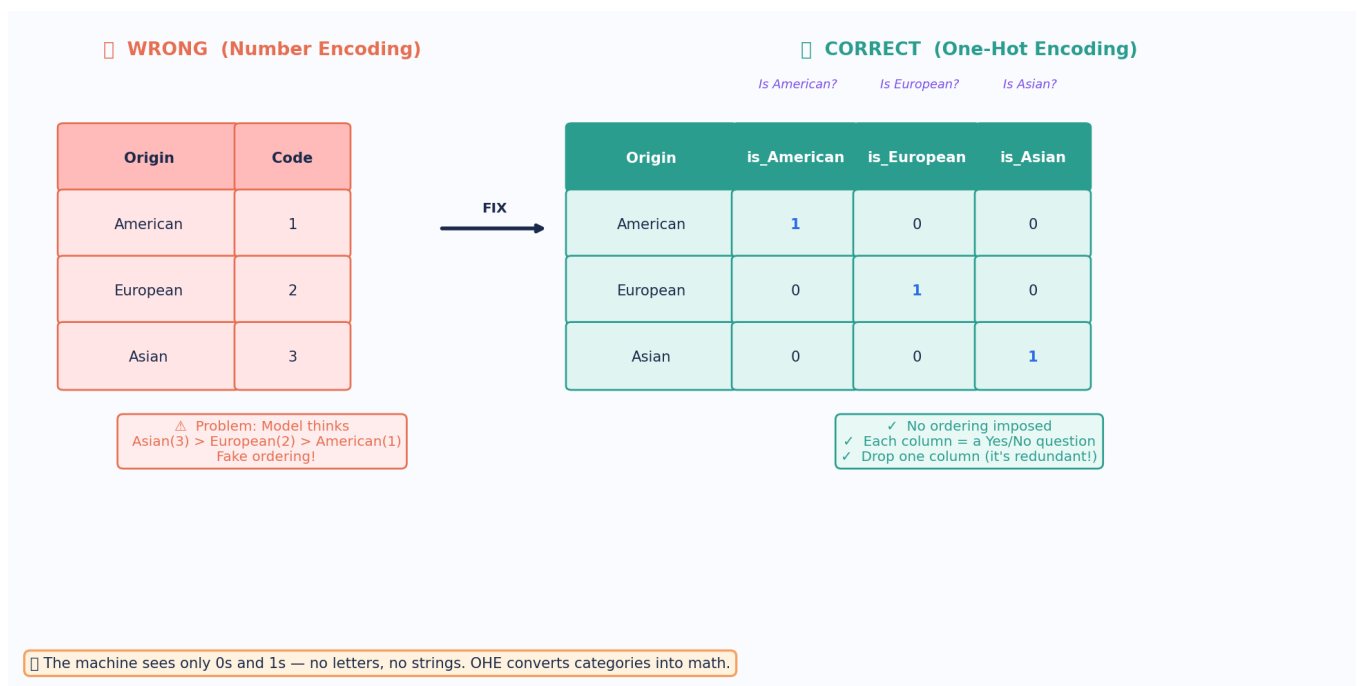


Figure 4: Why number encoding fails and how One-Hot Encoding fixes it

■ Always drop ONE dummy column (the "origin_europe" in our car dataset) to avoid the Dummy Variable Trap — where columns perfectly predict each other (multicollinearity).

■■ CONCEPT 4 — Handling Missing Values

In the auto-mpg dataset, 6 out of 398 rows had ? in the horsepower column — they were missing values in disguise. Real-world data is messy. How you handle gaps can significantly impact model quality.

Handling Missing Values — Strategies & When to Use Them

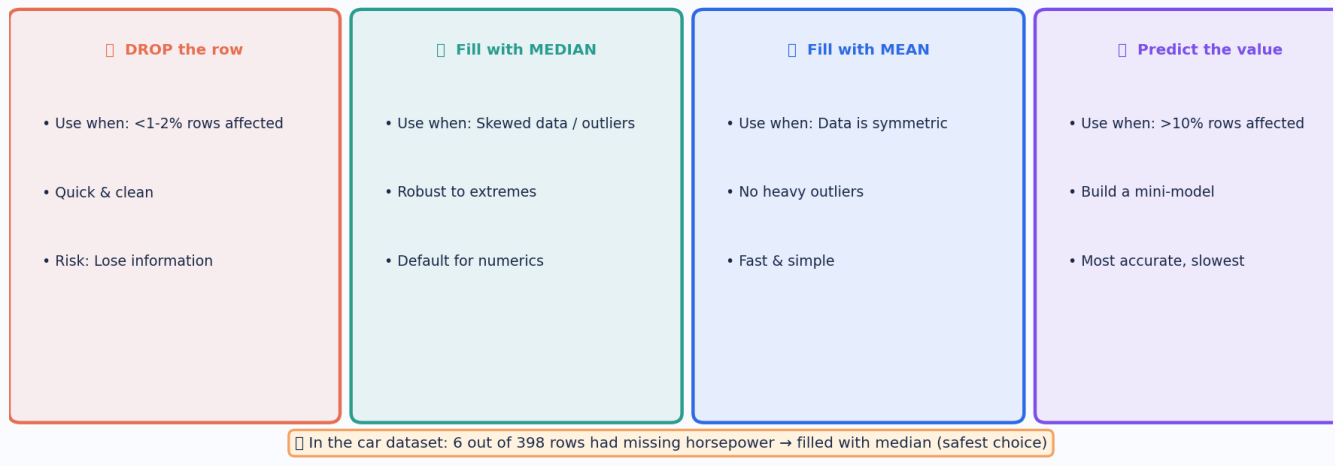


Figure 5: Four strategies for missing values — and when to use each

■ Why median over mean? In the car dataset, some cars have very high horsepower — the mean gets pulled upward. Median stays robust. Always check for skew before choosing your fill strategy!

■ CONCEPT 5 — Impact of Outliers on Regression

Least Squares regression minimizes the **sum of squared errors**. That "squared" part means a single very wrong prediction contributes massively to the total loss — the model will physically bend the line toward an outlier just to reduce that one huge error.



Figure 6: One outlier bends the entire regression line

■ Think of it like a tug-of-war rope. The outlier is a giant on one end — the regression line (rope) will always lean toward it. Remove outliers BEFORE fitting the model!

■ Also: outliers destroy Min-Max scaling! If one car has `km_driven = 500,000` (data entry error), all other values get squished into `[0, 0.1]`. Z-score scaling is more robust in such cases.

■ CONCEPT 6 — Multicollinearity & VIF

Linear regression assumes your input features are **independent** of each other. But what if *engine displacement* and *horsepower* are both measuring "engine power"? They carry the same information. Including both confuses the model — it can't figure out which one is responsible for the prediction, so the coefficients become unreliable.

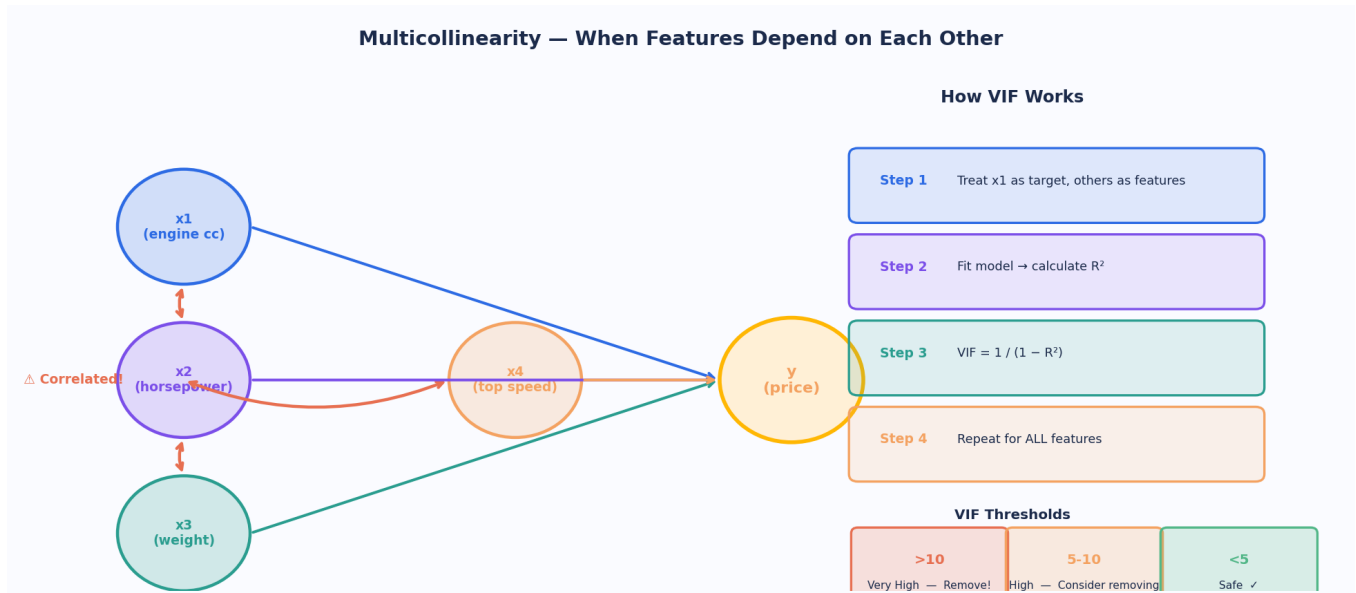


Figure 7: How VIF detects multicollinearity — treat each feature as a "dependent" variable and check if others can predict it

■ Real-world example in Cars24: "year" and "age" are perfectly correlated ($\text{age} = 2025 - \text{year}$). VIF for both will be infinity → remove one immediately!

■ CONCEPT 7 — The 5 Assumptions of Linear Regression

Linear regression is not a magic black box — it comes with **5 assumptions** that must hold for the results to be trustworthy. Violating them doesn't always break your model, but it does make the coefficients and p-values unreliable.

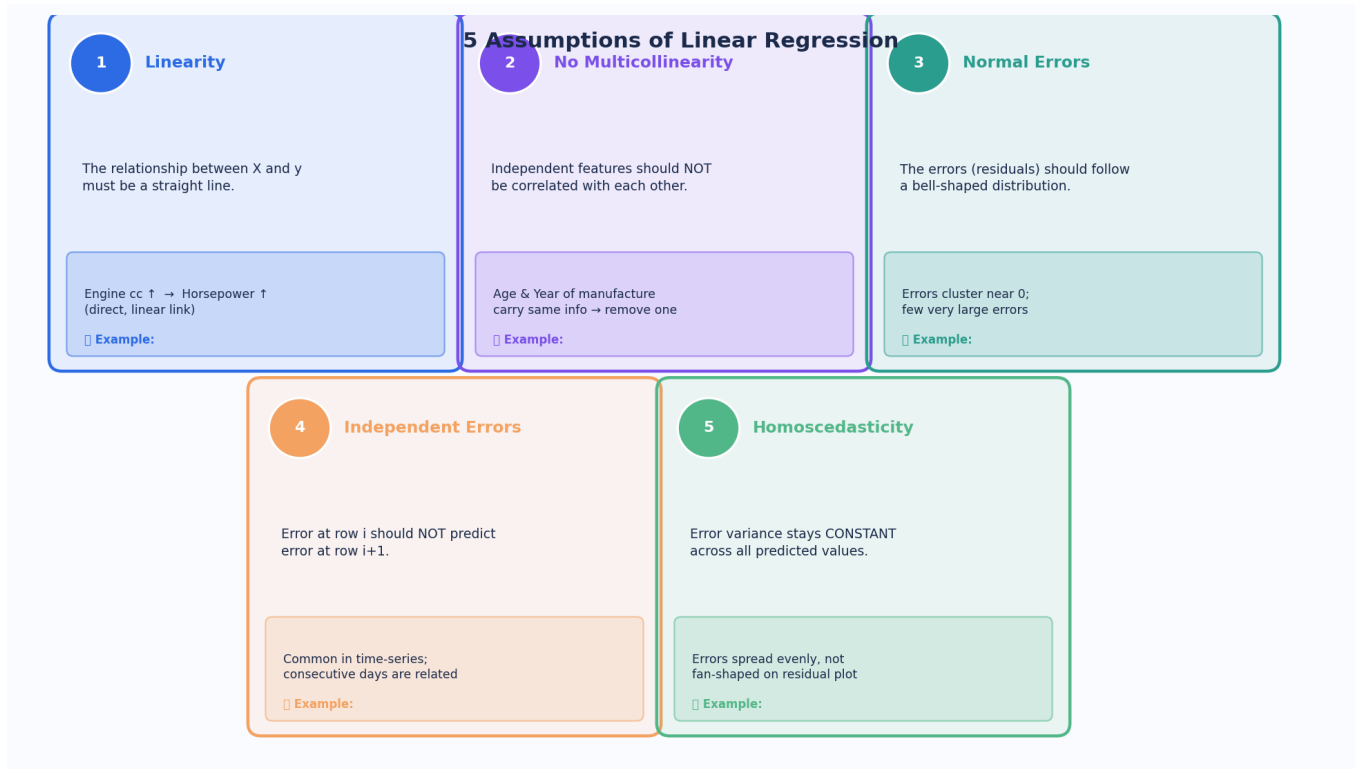


Figure 8: All 5 assumptions with real-world examples from the car dataset

■ Quick Checks: (1) Linearity → scatter plots X vs y. (2) Multicollinearity → $VIF > 5$. (3) Normal errors → histogram of residuals. (4) Independent errors → Durbin-Watson test. (5) Homoscedasticity → Goldfeld-Quandt test.

■ CONCEPT 8 — Polynomial Regression — When Lines Aren't Enough

US electricity consumption from 1960–2010 follows a **curved** pattern — it dips and then rises sharply. A straight line (degree 1) is hopeless here. Polynomial regression lets us fit curves by creating new features like x^2 , x^3 from the original feature x .

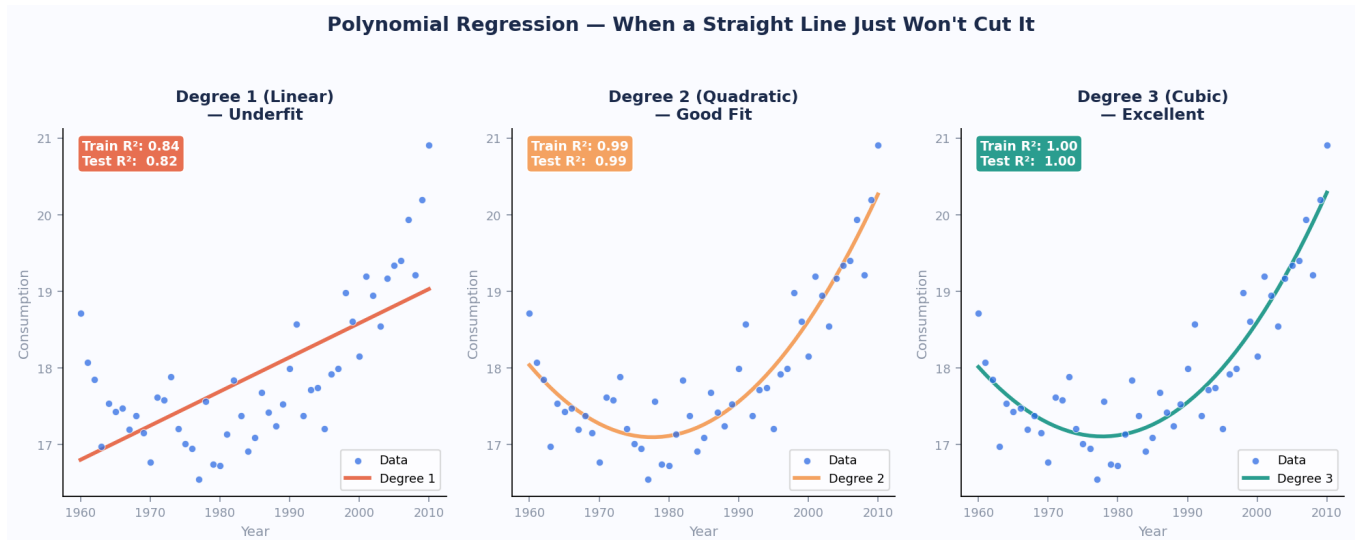


Figure 9: Degree 1 underfits, Degree 2 is good, Degree 3 is near-perfect — but watch test R^2 !

■ Occam's Razor: If Degree 2 gives $R^2=0.99$ and Degree 3 gives $R^2=0.998$ — pick Degree 2! The tiny gain is NOT worth the added complexity in production.

■ CONCEPT 9 — Bias-Variance Tradeoff

Every ML model makes errors. Those errors come from two sources: **Bias** (model too simple — misses patterns) and **Variance** (model too complex — memorises noise). You can't eliminate both simultaneously — reducing one increases the other.

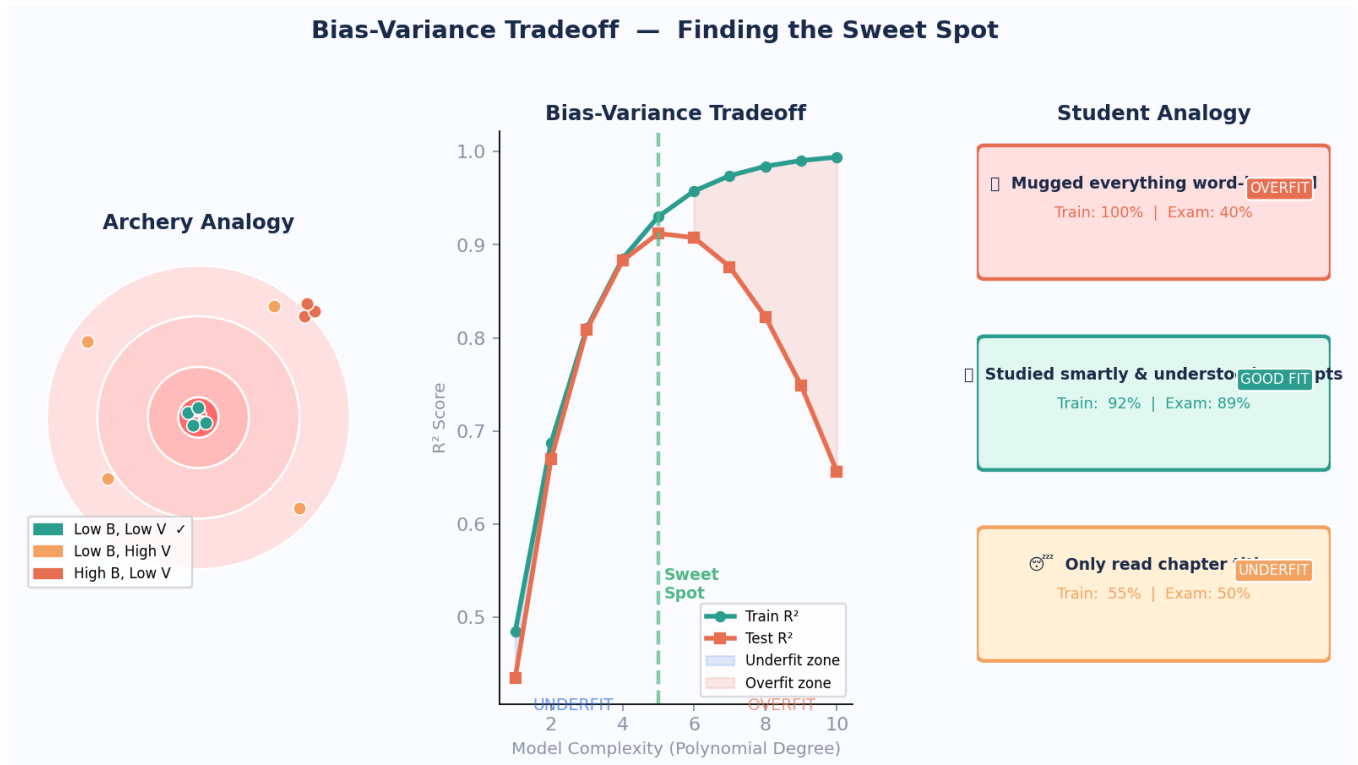


Figure 10: Archery analogy, R^2 tradeoff curve, and the student exam analogy

Scenario	Train R^2	Test R^2	Problem	Fix
Underfitting	Low	Low	High Bias	Add features / higher degree
Good Model	High	High	—	Keep it!
Overfitting	High	Low	High Variance	Regularization / more data

■ CONCEPT 10 — Regularization — Penalising Complexity

During training, the model tries to minimise error. To do this, it might make coefficients **extremely large**. Large weights = overfit model that is brittle on new data. Regularization adds a **penalty on large weights** to the loss function, forcing the model to stay simple.

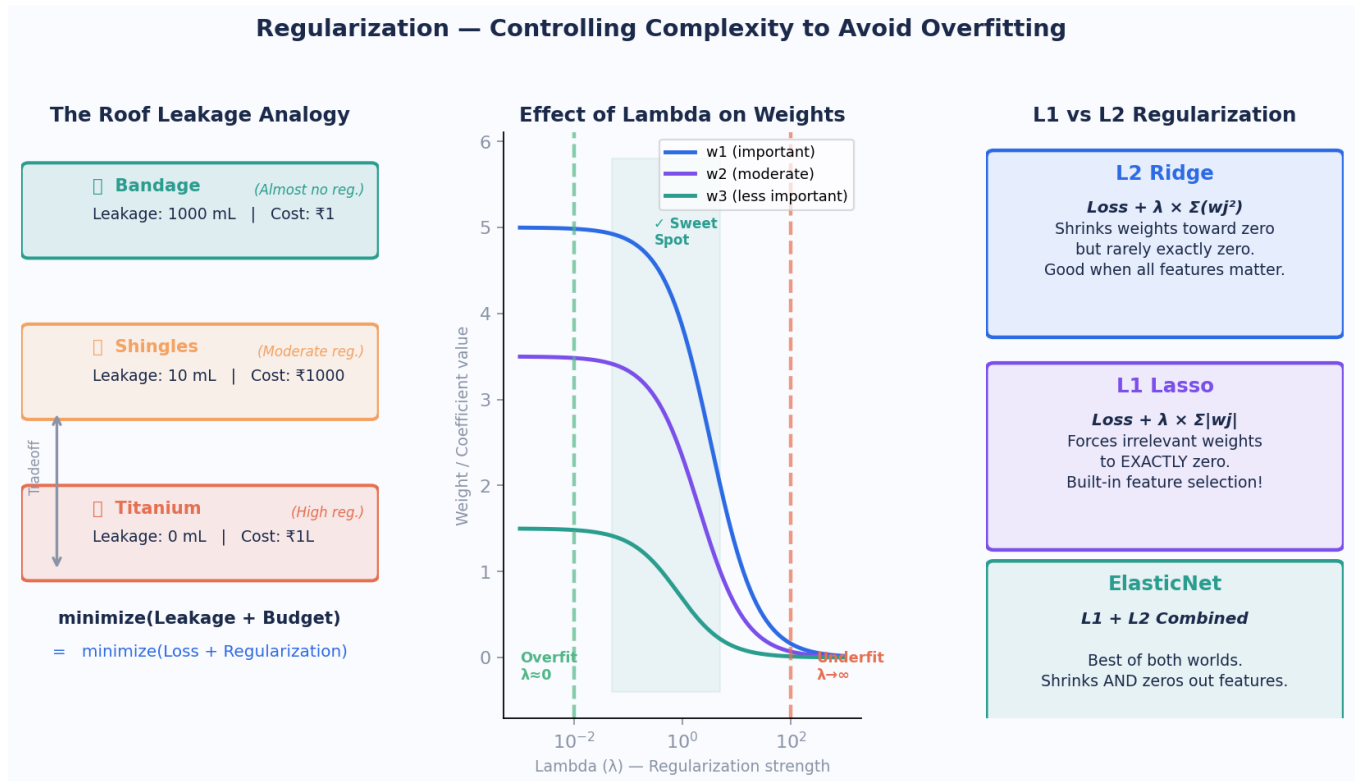


Figure 11: The roof leakage analogy, lambda effect, and L1 vs L2 comparison

Technique	Penalty Term	Key Property	sklearn class
Ridge (L2)	$\lambda \times \sum w^2$	Shrinks weights, none become 0	Ridge()
Lasso (L1)	$\lambda \times \sum w $	Sets irrelevant weights to 0!	Lasso()
ElasticNet	$\lambda \times \sum w + \lambda \times \sum w^2$	Shrinks AND zeros out features	ElasticNet()

■ Lambda (λ) is a hyperparameter — it controls the strength of regularization. $\lambda=0 \rightarrow$ no regularization (overfit risk). $\lambda \rightarrow \infty \rightarrow$ all weights $\rightarrow 0$ (underfit). Use cross-validation to find the ideal λ !

Quick Reference Card — Linear Regression Checklist

■	Split data	Use <code>train_test_split(random_state=42)</code> . Never test on train data.
■	Handle missing	Check dtypes. Replace ? with NaN. Fill with median (numeric) or mode (categorical).
■	Encode categories	<code>pd.get_dummies()</code> → drop one dummy column to avoid multicollinearity.
■	Remove outliers	Check box plots. Outliers pull regression lines and destroy scaling.
■	Scale features	MinMaxScaler or StandardScaler. Fit on train, transform both train+test.
■	Check VIF	Remove features with $VIF > 10$. Recheck until max $VIF < 5$.
■	Check assumptions	Linearity, No Multicollinearity, Normal errors, Independence, Homoscedasticity.
■	Compare degrees	Try degree 1, 2, 3 polynomial. Pick simplest model with good test R^2 .
■	Regularize	If overfit: try Ridge (L2) or Lasso (L1). Tune λ via cross-validation.
■	Report test R^2	Always report both train AND test R^2 . Big gap = overfitting.